38

BÖLÜM: ASENKRON BORU HATLARI & COMPLETABLEFUTURE | MODULE: ASYNCHRONOUS PIPELINES & COMPLETABLEFUTURE

Dönüşüm Boru Hatları: thenApply vs thenAccept vs thenRun & Thread Havuzu Geçişleri

Transformation Pipelines: thenApply vs thenAccept vs thenRun & Async Variants

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                DÖNÜŞÜM OPERATÖRLERİ KARŞILAŞTIRMASI          │
└──────────────────────────────────────────────────────────────┘

 Operatör          | Girdi Alır mı?   | Çıktı Üretir mi? | Fonksiyonel Arayüz
 ──────────────────┼──────────────────┼──────────────────┼──────────────────
 thenApply(f)      | EVET (T)         | EVET (U)         | Function<T, U>
 thenAccept(c)     | EVET (T)         | HAYIR (void)     | Consumer<T>
 thenRun(r)        | HAYIR (void)     | HAYIR (void)     | Runnable

 *Async Farkı:
 stage.thenApply(fn)      ──► Önceki görevi bitiren thread AYNI thread'de devam eder.
 stage.thenApplyAsync(fn) ──► Görevi Executor kuyruğuna atar, BAŞKA bir thread çalıştırır!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

1) thenApply: Aşçıya eti verirsin, pişirip biftek olarak geri verir (Girdi var, dönüşüm var). 2) thenAccept: Garsona bifteği verirsin, afiyetle yer veya müşteriye sunar, sana bir şey geri vermez (Girdi var, çıktı yok). 3) thenRun: İki işlem de bitince aşçı mutfaktaki gongu çalar; ne etle ilgilenir ne tabakla, sadece 'İş bitti!' der (Girdi yok, çıktı yok). Sonuna 'Async' eklersen bu işi başka bir personele devredersin!

🇬🇧 ENGLISH (ELI5)

1) `thenApply`: Handing raw flour to a baker and receiving bread back (consumes `T`, returns `U`). 2) `thenAccept`: Handing bread to a customer who consumes it without returning a value (consumes `T`, returns `void`). 3) `thenRun`: The kitchen clock ringing at closing time; independent of data (consumes `void`, returns `void`). Appending `*Async` (e.g., `thenApplyAsync`) dispatches execution to an Executor queue rather than running inline on the completing thread.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Senkron ve Asenkron varyantlar arasındaki kritik ayrım: `stage.thenApply()` çağrıldığında, eğer önceki aşama zaten tamamlanmışsa fonksiyon çağıran thread üzerinde çalışır. Önceki aşama henüz bitmemişse, o aşamayı tamamlayan iş parçacığı (worker thread) üzerinde inline olarak çalıştırılır; böylece gereksiz context-switch önlenir. `stage.thenApplyAsync(fn, executor)` ise işi kesinlikle belirtilen executor'ın iş kuyruğuna teslim eder. Uzun süren CPU hesaplamaları veya bağımsız thread context'i (örneğin MDC/SecurityContext) gerektiren adımlar için `*Async` varyantları tercih edilmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Inline Execution vs Async Dispatch: In `stage.thenApply(fn)`, if the preceding stage has already completed, `fn` executes synchronously on the calling thread. If pending, it executes inline on whichever worker thread completes the upstream stage, eliminating queue dispatch overhead. Conversely, `thenApplyAsync(fn, pool)` guarantees task submission into the designated executor's run-queue. Use `*Async` methods when the transform logic is CPU-intensive or requires clean thread isolation.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Pipeline Aşamaları ve Özel Executor Geçişi
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.Executors;

public class PipelineTransformationDemo {
    record UserProfile(long id, String rawName, String email) {}
    record EnrichedProfile(long id, String formattedName, boolean isVerified) {}

    void main() {
        try (var cpuPool = Executors.newFixedThreadPool(2);
             var ioPool = Executors.newVirtualThreadPerTaskExecutor()) {

            CompletableFuture.supplyAsync(() -> {
                // I/O işlemi: Veritabanından profili çek
                System.out.println("1. DB Fetch: " + Thread.currentThread());
                return new UserProfile(42L, "  ada lovelace  ", "ada@lovelace.org");
            }, ioPool)
            // Dönüşüm (Function): String temizleme & zenginleştirme (CPU Pool'a devret)
            .thenApplyAsync(user -> {
                System.out.println("2. CPU Transform: " + Thread.currentThread());
                return new EnrichedProfile(user.id(), user.rawName().trim().toUpperCase(), true);
            }, cpuPool)
            // Tüketim (Consumer): Bildirim gönder veya logla
            .thenAccept(enriched -> {
                System.out.println("3. Notification Sent: " + enriched);
            })
            // Tetikleme (Runnable): Metrik sayacını artır
            .thenRun(() -> {
                System.out.println("4. Pipeline tamamlandı, metrik güncellendi.");
            }).join(); // Test için ana thread'i beklet
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

thenApply içine asla bloklayan I/O yazmayın! Eğer bir önceki işlem hızlıca bitmişse, thenApply çağıran ana thread (örneğin Tomcat HTTP worker thread) üzerinde çalışıp ana isteği kilitler. Bloklayan her adım için thenApplyAsync ve dedicated I/O pool kullanın.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Never execute blocking I/O inside `thenApply`! If the upstream stage finished prior to registration, `thenApply` executes directly on the caller thread (e.g., HTTP request thread), causing catastrophic thread stalling. Reserve blocking operations for `thenApplyAsync`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• thenApply veri dönüştürür (T->U), thenAccept veriyi tüketir (T->void), thenRun sadece sinyal verir (void->void). *Async varyantları işi havuza devreder.

🎯 Key Takeaways for the Interview

• `thenApply` transforms data (T->U), `thenAccept` consumes (T->void), `thenRun` signals completion (void->void). `*Async` submits to an executor.